

简介

本应用手册具体介绍了在 FR30xx 系列芯片中，使用 GPIO 时的注意事项与使用方法。

旨在减少研发工程师的开发时间，提高效率，缩短项目周期。

目录

1. GPIO 概述	1
1.1. 普通控制单元	1
1.2. 低功耗控制单元	1
2. GPIO 的使用（普通控制单元）	2
2.1. 时钟配置	2
2.2. GPIO 初始化	2
2.3. 上下拉配置	3
2.4. 输入输出	4
2.5. 驱动能力	5
2.6. 外部中断	5
3. ADC 转换源（ADC ChanneMap）	6
4. GPIO 的使用（低功耗控制单元）	6
4.1. 时钟源	6
4.2. 输入输出	6
4.3. 上下拉配置	7
5. 睡眠唤醒	8
5.1. PortA~PortD 唤醒	8
5.2. PMU_IO 唤醒	10
6. 变更历史	12
7. 联系信息	13

1. GPIO 概述

FR30xx 系列芯片型号不同 GPIO 的数量也有所差异，但是 GPIO 的使用方法都是一致的，每个 GPIO 都可以复用成不同的功能，并且具有外部中断、睡眠唤醒等功能。GPIO 的控制权可以交由两个模块：普通控制单元和低功耗控制单元。在系统处于正常工作模式下时，这两个控制单元都可以用来控制 GPIO。当系统处于低功耗模式下时，普通控制单元处于掉电状态，这时只有低功耗控制单元可以控制部分 GPIO，下表列举所有 GPIO 的控制域及特点。

控制域	GPIO 范围	睡眠唤醒	睡眠时控制域状态
普通控制单元	PortA0~PortA15 PortB0~PortB15 PortC0~PortC15 PortD0~PortD15	支持，多个 GPIO 唤醒功能同时启用，唤醒时无法确定具体唤醒源。	掉电，锁存睡眠前的 GPIO 状态
低功耗控制单元	PMU_IO0~ PMU_IO7	支持，多个 GPIO 唤醒功能同时启用，唤醒时可以确定具体唤醒源。	不掉电，可以继续控制 GPIO。

1.1. 普通控制单元

在普通控制单元控制 GPIO 时，GPIO 可以用作通用输入输出口、外部中断口、复用串口 SPI 口、等常用外部接口功能。打开《FR30xx Specification》附录 I 可以查看可控制的 IO 口。需要注意的是可配置的 GPIO 中分了两块区域，高压域（3.3V）和低压域（1.8V），Flash、eMMC 等可以挂载在低压域以获得更高的同步时钟速率和更好的稳定性。下表列举最大数量 GPIO 型号芯片普通控制单元下的 GPIO 口。

电压域	GPIO 范围	上下拉电阻大小	驱动能力
高压（3.3V）	PortA0~PortA15 PortB0~PortB11 PortD0~PortD15	30K、60K	8mA、12mA、 16mA、24mA
低压（1.8V）	PortB12~PortB15 PortC0~PortC15	50K、100K	

注：上下拉电阻大小与驱动能力的配置在下文进行讲解。

1.2. 低功耗控制单元

低功耗控制单元提供了 8 个 PMU_I/O，由 PMU_IO0 ~ PMU_IO7 组成。在 SOC 进入低功耗模式时，PMU_IO 会持续工作，可以用来输出电平信号，或用来检测电平变化触发 PMU 中断，以唤醒 SOC。除了上述功能 PMU_IO0 ~ PMU_IO7 还可用作 ADC 的 ChannelMap[0]~ ChannelMap[7]。

2. GPIO 的使用（普通控制单元）

2.1. 时钟配置

普通控制单元时钟源可参考《FR30xx Specification》附录 III，时钟源配置完成后 PortA/B/C/D 共用同一个时钟源进行分频，但是每组 GPIO 需要单独使能。

```
__SYSTEM_GPIO_CLK_SELECT_COREH()

__SYSTEM_GPIO_CLK_SELECT_SPLL()

__SYSTEM_GPIO_CLK_DIV(__CLKDIV__)

__SYSTEM_GPIOA_CLK_ENABLE()

__SYSTEM_GPIOB_CLK_ENABLE()

__SYSTEM_GPIOC_CLK_ENABLE()

__SYSTEM_GPIOD_CLK_ENABLE()
```

2.2. GPIO 初始化

FR30xx 系列芯片继续延续 FR8000 SDK 的 GPIO 驱动代码，并且做了优化，更加灵活、更加简易，存放在 driver_gpio.c 中。本小段主要介绍 GPIO 的初始化配置，在完成初始化后才能使用下文的 GPIO 其他功能，调用 gpio_init 完成 GPIO 初始化操作：

1：调用前需要确定参数，GPIO 序号、模式、上下拉，如果配置成复用模式需要填写复用号。下面配置成复用模式 GPIO_MODE_AF_PP 进行示例。

```
/* GPIO mode

GPIO_MODE_INPUT / GPIO_MODE_INPUT_HRS / GPIO_MODE_OUTPUT_PP/ GPIO_MODE_AF_PP
```

```

    GPIO_MODE_EXTI_IT_LOW_LEVEL / GPIO_MODE_EXTI_IT_HIGH_LEVEL

    GPIO_MODE_EXTI_IT_FALLING / GPIO_MODE_EXTI_IT_RISING

*/

typedef struct
{
    uint32_t Pin;

    uint32_t Mode;

    uint32_t Pull;

    uint32_t Alternate;

}GPIO_InitTypeDef;

```

2: PortA8 和 PortA9 配置成复用成 UART2 的 TX 和 RX, 《FR30xx Specification》附录 I 查找到需要配置的复用号为 1 GPIO_FUNCTION_1。同时配置成上拉模式。

```

GPIO_InitTypeDef GPIO_Handle;

GPIO_Handle.Pin      = GPIO_PIN_8 | GPIO_PIN_9;

GPIO_Handle.Mode     = GPIO_MODE_AF_PP;

GPIO_Handle.Alternate = GPIO_FUNCTION_1;

GPIO_Handle.Pull     = GPIO_PULLUP;

gpio_init(GPIOA, &GPIO_Handle);

```

2.3. 上下拉配置

除了调用初始化接口还可以调用接口 `gpio_set_portpull` 进行上下拉配置, 参数分别为 GPIO 组号、GPIO 序号 (1bit 对应 1Pin)、上下拉选择, 下面进行对 PortA8 进行上拉配置示例。

```

typedef enum
{
    GPIO_NOPULL,      /*!< No Pull-up or Pull-down activation */

    GPIO_PULLUP,      /*!< Pull-up activation */
}

```

```
GPIO_PULLDOWN,    /*!< Pull-down activation */

}enum_Pull_t;

gpio_set_portpull(GPIOA, GPIO_PIN_8, GPIO_PULLUP);
```

2.4. 输入输出

对于 GPIO 的输入输出有两种方式，按 GPIO 组和 GPIO 序号。下面省略 GPIO 初始化配置，下面使用 PortA8 和 PortA9 按 GPIO 序号输入输出以及 PortA 按组输入输出的接口进行示例。

```
/* GPIO Bit SET and Bit RESET enumeration */

uint16_t pin_status;

typedef enum

{

    GPIO_PIN_CLEAR = 0u,

    GPIO_PIN_SET    = 1u,

}enum_PinStatus_t;

/* set PA1 value GPIO_PIN_SET */

gpio_write_pin(GPIOA, GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_SET);

/* set porta group value 0xFFFF */

gpio_write_group(GPIOA, 0xFFFF); //1bit~1pin

/* get porta group value */

pin_status = gpio_read_pin(GPIOA, GPIO_PIN_8 | GPIO_PIN_9);

/* get porta group value */

pin_status = gpio_read_group (GPIOA);
```

2.5. 驱动能力

为了适用不同的驱动场景，普通驱动单元可以配置 IO 的驱动能力，SDK 同样配备了相应的接口 `gpio_drive_current_config` 可以按 GPIO 序号进行配置，下面 PortA8 驱动能力配置为 16mA。

```
/* GPIO Drive Current Index */

typedef enum
{
    GPIO_DRIVE_8mA,

    GPIO_DRIVE_12mA,

    GPIO_DRIVE_16mA,

    GPIO_DRIVE_24mA,
}enum_GPIO_Drive_Current_t;

gpio_drive_current_config(GPIOA, GPIO_PIN_8, GPIO_DRIVE_16mA);
```

2.6. 外部中断

普通控制单元的 GPIO 可以配置 4 种外部中断触发类型：高电平触发、低电平触发、上升沿触发、下降沿触发。共 64 路，每个检测电路都可以分别配置。其中高电平触发、低电平触发可以设置电平触发信号硬件防抖。下面 PortA8 高电平触发进行示例：

1: GPIO 初始化，调用 `exti_set_Filter` 需要确定两个参数 `fu8_DIV`、`fu16_CNT`。

`fu8_DIV` 分频公式 $CLK = GPIO_CLK / (1 + PRE_VAL)$ ，`fu16_CNT` 代表防抖滤波计数值，当信号持续时间超过计数值时触发外部中断。

2: 调用 `exti_interrupt_enable` 使能外部中断。

3: 当外部中断线上检测到触发信号，进入中断服务函数调用 `exti_get_LineStatus` 确认是否获取到中断状态，最后调用 `exti_clear_LineStatus` 清除中断状态，退出中断服务函数等待下次触发信号来临。

```
/* set exti line filter */
```

```
exti_set_Filter (GPIOA, EXTI_LINE_0, 24, 100) ;

/* exti_interrupt_enable */

exti_interrupt_enable (GPIOA, EXTI_LINE_0) ;

NVIC_EnableIRQ (GPIOA_IRQn);

void gpioa_irq(void)
{
    if(exti_get_LineStatus (GPIOA, EXTI_LINE_0))
    {
        exti_clear_LineStatus (GPIOA, EXTI_LINE_0);
    }
}
```

3. ADC 转换源（ADC ChanneMap）

FR30xx 系列芯片 ADC 的 GPIO 复用与以往的芯片不同，使用时需要 ChanneMap，可选择外部转换源 PMU_IO4~ PMU_IO7 一个 4 个 GPIO。需要注意的是芯片低功耗后 ADC 控制单元将会掉电无法正常使用。ADC 相关的使用可以参考《FR30xx Specification》和 SDK 中的 Demo，由于本笔记主要介绍 GPIO 的使用，这里不再做过多讲解。

4. GPIO 的使用（低功耗控制单元）

4.1. 时钟源

FR30xx 系列芯片无论是否进入睡眠模式低功耗控制单元都会继续工作，在 SDK 中它的相关接口与普通控制单元分开，在 driver_pmu.c 中。时钟源为固定的低速时钟。

4.2. 输入输出

下面的示例展示如何将 PMU_IO0 配置成低功耗控制单元控制下的输入功能：

1: 配置成输出模式，输出高电平。

```
typedef enum
{
    PMU_GPIO_MODE_INPUT,

    PMU_GPIO_MODE_OUTPUT,
}enum_PMU_GPIO_MODE_t;

pmu_set_pin_dir(PMU_PIN_0, PMU_GPIO_MODE_OUTPUT);

pmu_set_pin_value(PMU_PIN_0, 1);
```

2: 配置输入模式，获取 GPIO 状态

```
typedef enum
{
    PMU_GPIO_MODE_INPUT,

    PMU_GPIO_MODE_OUTPUT,
}enum_PMU_GPIO_MODE_t;

pmu_set_pin_dir(PMU_PIN_0, PMU_GPIO_MODE_INPUT);

pmu_get_pin_value(PMU_PIN_0);
```

4.3. 上下拉配置

下面的示例展示如何配置 PMU_IO0 上下拉

```
typedef enum
{
    PMU_GPIO_NO_PULL,

    PMU_GPIO_PULL_UP,

    PMU_GPIO_PULL_DOWN,
}enum_PMU_GPIO_PULL_t;

pmu_set_pin_pull(PMU_PIN_0, PMU_GPIO_PULL_UP);
```

5. 睡眠唤醒

FR30xx 系列芯片在系统进入低功耗后有两种唤醒方式，普通控制单元下的 PortA~PortD 唤醒，低功耗控制单元下的 PMU_IO0 ~ PMU_IO7 唤醒。相关的 demo 可在 SDK 中的 examples\application\ble_simple_periphreal 中查看。

5.1. PortA~PortD 唤醒

在睡眠前先进行配置，然后才能启用睡眠唤醒功能：

- 1: PortA~PortD 需要配置成上下拉。
- 2: 使能 System 寄存器 Port_InputCutoffDisable 功能（高电平有效）。
- 3: 使能 GPIO 高压域与低压域电平检测中断，以及 PortA~PortD 睡眠唤醒中断。
- 4: 开启低功耗控制单元的 PMU 总中断。
- 5: 使能睡眠模式，锁存 PortA~PortD 睡眠前的状态。

当被锁存的 GPIO 状态发生变化时，退出睡眠状态同时触发 PMU 中断。进入睡眠模式后由于普通控制单元已经掉电所以无法判断究竟是哪个 IO 被唤醒，如果 IO 与外部芯片有连接，比如串口之类，在存在压差的情况下会有额外漏电。

下面展示 PortD 组的睡眠唤醒功能初始化，PortD 部分全部配置成上拉，使能 Port_InputCutoffDisable 功能、检测中断，开启睡眠唤醒中断与 PMU 总中断。

```
static void app_port_wakeup_demo_init(void)
{
    /* PortD Wakeup Init */

    SYSTEM->PortD_PullSelect = 0x0000ffff;

    SYSTEM->PortD_PullEN = 0x0000ffff;

    SYSTEM->PortD_InputCutoffDisable= 0x0000ffff;

    pmu_enable_isr(PMU_GPIO_GROUPH_INT_MSK_BIT | PMU_GPIO_GROUPL_INT_MSK_BIT);

    ool_write(PMU_GPIO_PMU_INT_MSK_BIT, 0x03);

    NVIC_EnableIRQ(PMU_IRQn);
}
```

```
}
```

Demo 中预留了睡眠唤醒后确认接口，睡眠唤醒后 GPIO 解锁在此接口中进行，解锁完成后才能重新配置 GPIO 的功能：

```
__RAM_CODE void user_entry_after_sleep (void)

{

    //其他代码

    ... ..

    ... ..

    //解锁

    ool_write(PMU_REG_PMU_GATE_M, ool_read(PMU_REG_PMU_GATE_M) & (~0x40));

    //解锁完成重新配置所需 GPIO 功能

    ... ..

    ... ..

}
```

除了睡眠唤醒后检查接口还预留了一个睡眠前确认接口，用于在睡眠前调用，GPIO 的状态锁存在这里进行，同时如果在下一次唤醒后继续睡眠时还需要 PortA~PortD 的睡眠唤醒功能，必须重新配置 Port_InputCutoffDisable 功能。

```
__RAM_CODE void user_entry_after_sleep (void)

{

    //其他代码

    ... ..

    ... ..

    // gpio_PD Wakeup Init

    SYSTEM->PortD_InputCutoffDisable = 0x0000ffff;

    //锁存

    ool_write(PMU_REG_PMU_GATE_M, ool_read(PMU_REG_PMU_GATE_M) | 0x40);

}
```

注：锁存功能并不只适用于睡眠唤醒，不使能睡眠唤醒，如果想要 PortA~PortD 睡眠后输出固定状态，也可以在配置完成之后进行锁存。

5.2. PMU_IO 唤醒

在睡眠前先进行配置，然后才能启用睡眠唤醒功能：

1: PMU_IO0 ~ PMU_IO7 需要配置成上下拉。

3: SDK 中配备了 PMU_IO 唤醒的完整配置接口，调用 `pmu_gpio_int_init` 进行初始化，填写需要检测哪个 PMU_IO、默认上下拉状态、引脚引脚监控状态参考值三个参数，对应 PMU_IO 的引脚监控中断使能也会在此接口中实现。

5: 调用 `pmu_enable_isr` 开启 PMU_IO 中断，开启 PMU 总中断。

6: 使能睡眠模式。

如果引脚引脚监控状态参考值为 0，监测的 PMU_IO 由高变低触发 PMU 中断，芯片退出低功耗状态。如果如果引脚引脚监控状态参考值为 1，监测的 PMU_IO 由低变高触发 PMU 中断，芯片退出睡眠状态。

下面的示例展示 demo 中如何配置 PMU_IO0 部分的睡眠唤醒功能

```
/* PUM_IO0 Wakeup init */

static void app_gpio_wakeup_demo_init(void)
{
    pmu_gpio_int_init(PMU_PIN_0, PMU_GPIO_PULL_UP, 0);

    /* PUM_IO ISR Enable */

    pmu_enable_isr(PMU_GPIO_PMU_INT_MSK_BIT);

    /* PUM ISR Enable */

    NVIC_EnableIRQ(PMU_IRQn);
}
```

配置完上述步骤后开启睡眠模式，当检测到引脚电平变化时会进入 PMU 中断，退出睡眠模式。

```
void PMU_GPIO_PMU_IRQHandler(void)
```

```
{  
  
    uint16_t data = ool_read16(PMU_REG_PIN_DATA);  
  
    uint16_t result = ool_read16(PMU_REG_PIN_XOR_RESULT);  
  
    /* update last value with latest data */  
  
    ool_write16(PMU_REG_PIN_LAST_V, data);  
  
    /* clear last XOR result */  
  
    ool_write16(PMU_REG_PIN_XOR_CLR, result);  
  
}
```

6. 变更历史

版本号	日期	更新内容
V1.0	2023.9.16	首版

7. 联系信息

公司：上海富芮坤微电子有限公司

地址：中国(上海)自由贸易试验区碧波路 912 弄 8 号 501-A 室

电话：+86-21-5027-0080

Website: www.freqchip.com

Sales Email: sales@freqchip.com

本文档的所有部分，其著作权归上海富芮坤微电子有限公司（简称富芮坤）所有，未经富芮坤授权许可，任何个人及组织不得复制、转载、仿制本文档的全部或部分。富芮坤保留在不另行通知的情况下随时对产品或本文档进行更改、修正、增强的权利。购买者应在订购前获得富芮坤产品的最新相关资料。